

Тренажёр набора текста

Пособие для понимания кода и защиты проекта

C# и Avalonia с нуля

Здесь разобрано, что делает приложение, где находятся его основные части и как объяснить их преподавателю. Сначала изучи базовые понятия, затем проследи действия пользователя по коду. Для защиты важнее понимать причины каждого шага, чем помнить все строки наизусть.

Как пользоваться пособием

Первый проход — основы C# и Avalonia. Второй — устройство именно этого приложения с открытыми рядом исходниками. Третий — вопросы, речь и демонстрация. Короткие примеры кода можно читать как перевод обычного действия на язык программы.

Если до защиты остался час, прочитай основной сценарий, архитектуру, ключевые алгоритмы и ограничения, затем проговори речь и выполни план демонстрации. После этого вернись к незнакомым словам в начальных разделах.

Сведения	Значение
Приложение	Тренажёр набора текста
Исходники	TypingTrainerApp.slnx
Дата проверки	11 сентября 2026
Автоматические тесты	15 пройдено

1 C sharp и платформа dot NET с нуля

Язык платформа и интерфейс

C# читается «си шарп». Это язык, на котором мы описываем действия программы: прочитать текст, сравнить символы, обработать нажатие кнопки, сохранить файл. Код в файле .cs — обычный текст с правилами записи. Компьютеру нужно перевести его в выполняемую форму.

.NET читается «дот нет». Это платформа: среда выполнения и большой набор готовых средств для работы со строками, файлами, временем и коллекциями. C# — язык инструкций, .NET — средства для их выполнения. Avalonia — библиотека для построения окон и элементов интерфейса поверх .NET. Это три связанные, но разные вещи.

Представь мастерскую. C# — язык, на котором написано задание мастеру. .NET — оборудование и общие инструменты. Avalonia — набор деталей для панели управления: окна, кнопки, поля ввода. Проект приложения соединяет всё это и добавляет собственные правила работы.

SDK — набор для разработчика. В нём есть инструменты сборки, компилятор и средства запуска. Runtime — среда выполнения готового приложения. Для изменения и сборки нужен SDK. У готовой самостоятельной сборки необходимые компоненты .NET могут лежать рядом с приложением.

NuGet — система получения библиотек. PackageReference в .csproj указывает имя и версию готового пакета. ProjectReference связывает проекты внутри решения. Например, приложение использует классы своего Core через ProjectReference. Это не копирование текста исходников вручную.

Как исходники становятся работающей программой

Обычная последовательность такая: исходники C# и разметка Avalonia → сборка проекта → файлы приложения → запуск среды .NET → создание окна → ожидание действий пользователя. В обычной сборке C# сначала получается промежуточный код IL; среда выполнения может преобразовывать его в машинный код через JIT. Для объяснения этих проектов достаточно знать этот путь; отдельная AOT-сборка здесь не показана в настройках проектов.

Программа с окном не читает весь код сверху вниз заново после каждого клика. При запуске она создаёт объекты и подписывает обработчики на события. Затем работает цикл обработки событий: нажали кнопку — вызван соответствующий метод; изменили текст — вызван другой метод.

Файлы .csproj описывают отдельные проекты. Решение .sln объединяет несколько проектов для удобной работы. Каталог bin содержит результаты сборки, obj — промежуточные файлы. Правильное поведение приложения следует в исходниках, а не в сгенерированных файлах этих каталогов.

Переменная и тип данных

Переменная — именованное место для значения. Тип отвечает на вопрос «какие значения здесь допустимы и какие операции с ними можно выполнять». Имя отвечает на вопрос «зачем это значение понадобилось».

```
int errors = 2;
double accuracy = 97.5;
bool isActive = true;
string text = "Привет";
char letter = 'П';
```

int хранит целое число; double — число с дробной частью; bool — true или false; string — строку; char — одну кодовую единицу UTF-16. В обычном русском и английском тексте отдельная буква обычно занимает один char. Однако эмодзи и некоторые сложные символы могут занимать несколько единиц: String.Length не является универсальным счётчиком видимых знаков.

Знак = присваивает значение. Знак == сравнивает два значения. Поэтому errors = 2 означает «записать 2», а errors == 2 — «проверить, равно ли значение двум». Точка с запятой завершает многие инструкции. Строки пишутся в двойных кавычках, отдельные char — в одинарных.

```
var count = 10;
var caption = "Готово";
```

var не означает «любой тип». Компилятор выводит тип из правой части: count будет int, caption — string. После этого нельзя просто записать строку в count. Такой вывод типа сокращает запись, сохраняя проверку типов.

Условия циклы и индексы

```
if (errors == 0)
{
    return "Без ошибок";
}
return "Есть ошибки";
```

Это учебный фрагмент тела метода, возвращающего string. if означает «если». Условие в круглых скобках должно дать bool. Фигурные скобки объединяют инструкции. return возвращает результат и сразу прекращает текущий вызов метода. В методе void можно написать просто return, чтобы выйти без значения.

```
for (var i = 0; i < text.Length; i++)
{
    var symbol = text[i];
}
```

for повторяет действия. i = 0 — старт; i < text.Length — условие продолжения; i++ — увеличение на единицу после прохода. Индексы начинаются с нуля: в строке «кот» text[0] — «к», text[1] — «о», text[2] — «т». Обращение за пределы строки вызывает ошибку. foreach перебирает готовые элементы коллекции без ручного управления индексом.

Частые операторы: != — не равно; !flag — противоположное логическое значение; && — одновременно выполнены оба условия; || — выполнено хотя бы одно. В выражении a && b правая часть не вычисляется, если a уже false. Тернарная запись condition ? yes : no выбирает одно из двух значений.

Класс объект поле и свойство

Класс — описание устройства определённой сущности. Объект — конкретный экземпляр такого класса в памяти. Один класс настроек позволяет создать несколько объектов с разными настройками. new создаёт экземпляр, а конструктор задаёт его начальное состояние.

```
public sealed class TrainerSettings
{
    public float FontSize { get; set; } = 16f;
    public bool ShowKeyboard { get; set; } = true;
}
```

Это фрагмент реального класса тренажёра. class объявляет класс. public делает его доступным другим частям программы. sealed запрещает создавать классы-наследники от него. FontSize и ShowKeyboard —

свойства. `get` разрешает чтение, `set` — изменение. `= 16f` задаёт начальное значение `float`; суффикс `f` уточняет тип числа. У свойства `bool` без явного начального значения исходное значение `false`.

```
var settings = new TrainerSettings();
settings.FontSize = 24;
```

Первая строка создаёт объект. Вторая меняет его свойство. Точка означает обращение к члену объекта. Поле хранит данные непосредственно внутри класса; свойство предоставляет доступ к данным и при необходимости выполняет дополнительную логику. Поэтому свойство может не хранить отдельное значение, а вычислять его при чтении.

```
public bool IsDirty => text != savedText;
```

«это учебное вычисляемое свойство: это учебное вычисляемое свойство: это учебное вычисляемое свойство: это учебное вычисляемое свойство: «есть несохранённые изменения, если текущий текст отличается от сохранённого». `=>` здесь сокращает запись тела свойства. У свойства нет `set`: присвоить ему `true` вручную нельзя. Нужно изменить `text` или `savedText`, и результат сравнения изменится.

`private` разрешает обращаться к члену только внутри содержащего типа. `public` открывает доступ извне. `internal` ограничивает доступ сборкой. `readonly` запрещает переписывать поле после допустимой инициализации; но если поле содержит список, это само по себе не запрещает добавлять элементы в список. `const` задаёт константу времени компиляции.

Метод и конструктор

Метод — именованное действие. Параметры — входные данные, возвращаемое значение — результат. Вызов `session.Start(text, true)` означает «попросить объект `session` начать попытку с данным текстом и учётом регистра». Тип `void` означает, что метод не возвращает значение.

```
public void Reset()
{
    IsActive = false;
    IsComplete = false;
}
```

Это сокращённый учебный фрагмент сброса состояния. `Reset` — имя метода, пустые круглые скобки означают отсутствие параметров. В настоящем методе тренажёра также очищаются строки и время. Конструктор носит имя класса, не имеет типа возвращаемого значения и вызывается при `new`.

В коде встречается современная запись конструктора прямо возле имени класса: `TypingSession(TimeProvider? timeProvider = null)`. Это первичный конструктор. Параметр со значением по умолчанию разрешено не передавать. Он нужен для выбора источника времени, в том числе тестового.

Наследование интерфейсы и основы ООП

ООП — способ организовать программу вокруг объектов, их данных и действий. На защите полезно объяснить три идеи на конкретных примерах. Инкапсуляция: внутреннее состояние меняется через методы и свойства с контролируемым доступом. Наследование: `MainWindow : Window` расширяет готовое окно `Avalonia`. Полиморфизм: код может работать с разными реализациями одного интерфейса через общий договор.

Интерфейс перечисляет возможности, которые обязан предоставить объект. `IAppDialogs` описывает выбор файла и вопросы пользователю. Настоящий `AppDialogs` показывает окна, а `FakeDialogs` в тестах заранее возвращает ответы. `ViewModel` обращается к договору `IAppDialogs`, поэтому его легче проверять без ручных кликов.

Композиция означает, что объект содержит другие объекты: MainWindow хранит TypingSession или ViewModel. Здесь обычно не нужно наследовать окно от «сессии»; окно использует сессию. Это хороший пример выбора отношения «имеет» вместо отношения «является».

override заменяет разрешённую виртуальную реализацию базового класса. Например, App переопределяет Initialize. base обращается к базовой реализации. partial означает, что определение класса разделено на части, в том числе исходную и сгенерированную из разметки.

Коллекции и LINQ

List<T> — изменяемый список элементов типа T. List<TypingSessionStats> хранит результаты попыток. Dictionary<TKey, TValue> сопоставляет ключу значение: Dictionary<char, Border> позволяет найти изображение клавиши по символу. Здесь Dictionary — структура данных C#, а WordDictionary — собственный класс приложения, несмотря на похожие названия.

IReadOnlyList<T> предоставляет чтение списка через данный интерфейс. Queue<T> — очередь: первый добавленный элемент извлекается первым. Она используется при обходе папок. Запись [] в подходящем контексте создаёт пустую коллекцию. Угловые скобки задают параметр типа; это называется обобщением, или generics.

LINQ — набор операций над последовательностями. Where отбирает элементы, Select преобразует каждый элемент, OrderByDescending сортирует по убыванию, Take оставляет первые N, ToList материализует результат в список. Max находит максимум, Average — среднее, GroupBy собирает элементы в группы.

```
statistics.OrderByDescending(item => item.Date)
    .Take(500)
    .ToList();
```

Это цепочка из тренажёра. Сначала самые новые результаты оказываются впереди, затем сохраняются только первые 500, затем получается отдельный список. item => item.Date — лямбда: маленькая функция «для данного элемента взять его дату». => может обозначать лямбду или краткое тело члена класса; смысл определяется контекстом.

Пустое значение и короткие записи

null означает отсутствие объекта или значения ссылки. Пустая строка "" существует и имеет длину ноль; null означает, что строки нет. string? показывает, что отсутствие допустимо. Настройка Nullable в .csproj помогает компилятору предупреждать об опасных обращениях.

InputBox.Text ?? "" читается «взять Text, а если там null, использовать пустую строку». Выражение document?.Path читает Path только при наличии document, иначе даёт null. Знак ! после выражения, например Inlines!, подавляет предупреждение компилятора о null, но не создаёт объект и не защищает во время выполнения.

is проверяет тип или шаблон. is not WordDictionary text означает «если значение не удалось распознать как WordDictionary и получить переменную text». switch выбирает ветку по значению или типу. nameof(IsDirty) даёт строку с именем члена; при переименовании инструменты могут обновить ссылку безопаснее, чем обычный текст в кавычках.

\$"Ошибок: {errors}" вставляет значение в строку. Формат :F1 показывает одну цифру после десятичного разделителя, :F0 — число без дробной части. Это меняет отображение, а не сам расчёт. \n в строке означает перевод строки. // начинает комментарий для человека.

События асинхронность и освобождение ресурсов

Событие — уведомление о том, что что-то произошло. Обработчик — код, который реагирует. Запись `button.Click += ...` подписывает обработчик; `-=` снимает подписку. Подписка не нажимает кнопку немедленно. В выражении `(_, _) => StartSession()` параметры события не используются, а после события вызывается `StartSession`.

`async` и `await` позволяют описать ожидание операции без блокирующего ожидания потока в точке незавершённого `await`. `Task` представляет выполняемую или завершённую операцию; `Task<string>` дополнительно даёт строковый результат. `async` сам по себе не создаёт отдельный поток. `Task.Run` обычно отправляет вычислительную работу в пул потоков.

```
var upperText = await Task.Run(  
    () => text.ToUpperInvariant()  
);
```

Учебный пример: обработка текста запускается в фоне, а после ожидания результат используется для обновления интерфейса. Элементы Avalonia создают и меняют в UI-потоке — потоке интерфейса. `Dispatcher` помогает выполнить работу в этом потоке. Нельзя считать, что любую операцию с кнопками допустимо делать внутри `Task.Run`.

Исключение — сигнал о невозможности нормально выполнить действие, например прочитать недоступный файл. `try` содержит попытку; `catch` обрабатывает подходящую ошибку; `finally` выполняет завершающие действия при выходе из блока. `throw` создаёт или передаёт исключение дальше. Ловить исключения полезно там, где можно объяснить проблему пользователю или восстановить состояние.

`using` в начале файла подключает пространство имён для коротких имён типов. `using var stream = ...` внутри метода имеет другой смысл: освободить ресурс при выходе из области действия. `IDisposable` и `Dispose` нужны, например, для потоков файлов и изображений. Сборщик мусора освобождает управляемую память, но своевременное закрытие файла всё равно требует правильного управления ресурсом.

2 Avalonia и устройство окна

Что описывает AXAML

Avalonia — средство создания графического интерфейса. Файл `.axaml` использует синтаксис XML для описания дерева элементов. Это декларативная запись: мы говорим, какие элементы нужны и как они размещены. Код `.axaml.cs` описывает связанные с окном действия. Их связывает `x:Class`, а загрузка разметки выполняется при `InitializeComponent`.

```
<Button x:Name="StartButton"  
    Content="Начать" />
```

`Button` — кнопка. `x:Name` — имя для обращения из C#. `Content` — содержимое кнопки. Запись `/>` закрывает элемент без вложенных элементов. `xmlns` объявляет пространство имён XML: оно помогает понять, из какой библиотеки пришёл тип. URL в `xmlns` не означает, что приложение загружает разметку с сайта при каждом запуске.

`Window` — окно; `Grid` — сетка; `StackPanel` размещает элементы последовательно; `TextBlock` показывает текст; `TextBox` принимает ввод; `Button` выполняет действие; `ComboBox` предлагает выбор; `CheckBox` хранит состояние флажка; `ScrollView` даёт прокрутку. `Border` добавляет фон, рамку и внутренние отступы. `TreeView` отображает дерево, `TabControl` — вкладки.

Как работает размещение

```
<Grid ColumnDefinitions="*,340">
    <TextBlock Text="Рабочая область" />
    <TextBlock Grid.Column="1" Text="Настройки" />
</Grid>
```

Учебный пример повторяет принцип тренажёра: справа фиксированная ширина 340, слева оставшееся место. * означает долю свободного пространства, Auto — размер по содержимому, число — фиксированный размер в логических единицах интерфейса. Столбцы и строки нумеруются с нуля. Поэтому Grid.Column="1" означает второй столбец.

Margin — внешний отступ, Padding — внутренний отступ между границей и содержимым. IsVisible скрывает или показывает элемент; IsEnabled разрешает или блокирует взаимодействие; IsReadOnly у TextBox запрещает редактирование. TextWrapping="Wrap" переносит длинные строки. Минимальные размеры окна задаются через MinWidth и MinHeight.

Styles задают повторяющееся оформление по селектору. Например, стиль Button меняет все подходящие кнопки, а Classes="tool" позволяет выделить отдельную группу. FluentTheme предоставляет базовый внешний вид элементов. Тема не отвечает за алгоритмы сравнения текста или сохранения файлов.

Привязка данных и MVVM

Binding связывает свойство элемента с данными. DataContext — объект, относительно которого обычно ищут свойства привязки. Если DataContext = ViewModel, поэтому {Binding WindowTitle} получает заголовок из ViewModel. x:DataType указывает ожидаемый тип данных для разметки, но сам по себе не создаёт объект и не назначает DataContext.

```
<TextBox Text="{Binding DocumentText, Mode=TwoWay}" />
```

TwoWay означает двустороннюю связь: ввод в поле меняет DocumentText, а изменение DocumentText в модели отражается в поле. Для обновления после изменения обычного свойства модель посылает PropertyChanged через интерфейс INotifyPropertyChanged. Без такого уведомления значение в памяти может измениться, а отображение остаться прежним.

MVVM расшифровывается как Model, View, ViewModel. Model содержит данные и правила предметной области; View — отображение; ViewModel — состояние, свойства и команды, с которыми работает View. Это способ разделить обязанности, а не обязательная библиотека. Наличие .axaml и .axaml.cs ещё не доказывает использование MVVM.

Команда ICommand предоставляет Execute для выполнения и CanExecute для проверки доступности. Один объект команды можно привязать к меню, кнопке и сочетанию клавиш. Code-behind при этом допустим для деталей самого интерфейса: фокус, положение курсора, выделение, закрытие окна.

Путь запуска приложения

Program.Main вызывает BuildAvaloniaApp. AppBuilder.Configure<App>() выбирает класс приложения, UsePlatformDetect() подключает подходящую платформенную реализацию, LogToTrace() включает вывод диагностических сообщений в средства трассировки. StartWithClassicDesktopLifetime запускает жизненный цикл настольного приложения.

App.Initialize загружает App.axaml с темой и стилями. OnFrameworkInitializationCompleted создаёт MainWindow и назначает его главным окном. Конструктор MainWindow загружает разметку, готовит данные

и подписки на события. Затем окно ожидает действий пользователя. Атрибут [STAThread] задаёт модель главного потока, нужную в том числе для совместимости с некоторыми механизмами Windows.

Avalonia позволяет строить приложения для разных платформ, однако эти проекты настроены как настольные приложения. Сам факт применения Avalonia не означает, что уже создана и проверена версия для Android или iOS. Проверки в этом пособии относятся к данному окружению Windows.

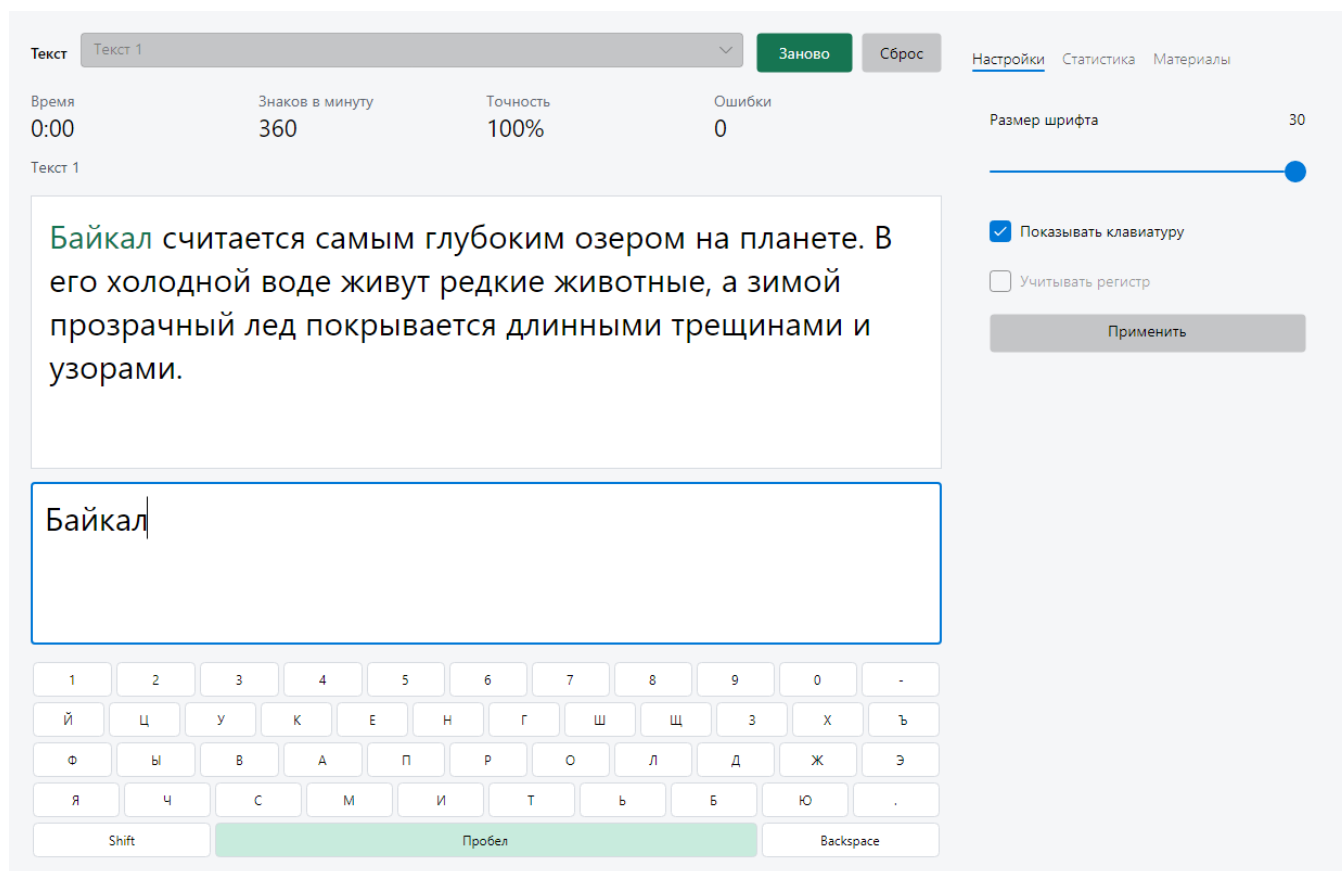
3 Как устроен тренажёр набора текста

Задача приложения и основной сценарий

Тренажёр даёт образец текста, принимает ввод пользователя, сравнивает его с образцом и показывает время, скорость, точность и ошибки. После завершения попытка попадает в историю. Пользователь может поменять размер текста, скрыть подсказку-клавиатуру и включить сравнение с учётом регистра.

Здесь заранее подготовлены десять текстов: семь русских «Текст 1» — «Текст 7» и три английских «Text 8» — «Text 10». В текущем интерфейсе нет создания собственных текстов. Каталог встроен в AppDataStore.CreateDefaultTexts, а не загружается из внешней базы данных.

Последовательность пользователя: выбрать текст → при необходимости настроить приложение и нажать «Применить» → нажать «Начать» → перепечатать образец → посмотреть итог → открыть вкладку «Статистика». «Сброс» прекращает текущую попытку без добавления результата. «Заново» запускает попытку сначала.



На изображении показан интерфейс из автоматического теста. Слева находятся выбор текста, показатели, образец, поле ввода и подсказка-клавиатура. Справа — вкладки «Настройки», «Статистика» и «Материалы». Цвет символов образца показывает результат сравнения.

Карта файлов и зависимостей

Решение называется TypingTrainerApp.slnx. Внутри три проекта. TypingTrainer — исполняемое приложение с окнами; TypingTrainer.Core — логика и данные без зависимости от Avalonia; TypingTrainer.Tests — автоматические проверки. Такая граница позволяет проверять формулы отдельно от окна.

Файл или проект	Что искать в нём
TypingTrainer/Program.cs	Вход в программу и настройку Avalonia
TypingTrainer/App.axaml и App.axaml.cs	Общую тему и создание главного окна
TypingTrainer/MainWindow.axaml	Расположение кнопок полей и вкладок
TypingTrainer/MainWindow.axaml.cs	Обработчики событий и обновление экрана
TypingTrainer.Core/Models.cs	Объекты текста настроек и результата
TypingTrainer.Core/TypingSession.cs	Начало попытки сравнение и расчёт показателей
TypingTrainer.Core/AppDataStore.cs	Встроенные тексты и запись JSON
TypingTrainer.Tests	Проверки логики и окна
typing-trainer-notebook.ipynb	Отдельный анализ накопленных результатов на C#

Зависимость направлена от окна к Core. TypingSession не знает, какого цвета кнопка и как называется TextBox. Главное окно передаёт сессии строку, получает статистику и решает, как её показать. AppDataStore отдельно занимается чтением и записью данных.

Фактическая архитектура

Для этого приложения корректное описание — событийный интерфейс с code-behind и вынесенным ядром. Главный управляющий код находится в MainWindow.axaml.cs. Полноценного отдельного ViewModel для окна нет. В разметке есть отдельные Binding для элементов списков, но они не превращают весь проект в MVVM.

Движение данных: пользователь → InputBox.TextChanged → MainWindow.OnInputChanged → TypingSession.UpdateInput → TypingSession.GetStatistics → обновление надписей и цвета. При завершении подключается AppDataStore.SaveStatistics, который записывает результат на диск.

Плюс такого решения — прямой и понятный путь от кнопки к действию. Минус — MainWindow выполняет много обязанностей, и при росте приложения его сложнее сопровождать. Возможное развитие — вынести состояние и команды в ViewModel, сохранив TypingSession и AppDataStore. Это предложение по развитию, а не уже реализованная часть.

Три класса с данными

WordDictionary хранит Id, Name, Words и Text. Id — идентификатор Guid; Name — имя в списке; Text — полный образец. Words содержит разбиение на слова, но основная проверка набора использует Text. Название WordDictionary сохранилось, хотя фактически пользователь выбирает готовый текст, а не словарь для случайной генерации.

TrainerSettings хранит FontSize, ShowKeyboard и CaseSensitive. По умолчанию размер 16, клавиатура видна, регистр не учитывается. TypingSessionStats хранит дату, название текста, количество введённых и правильных символов, ошибки, секунды, точность и скорость.

В TypingSessionStats свойство WordsPerMinute хранит скорость в правильно набранных знаках в минуту. Это историческое имя поля JSON, сохранённое ради совместимости. Свойство CharactersPerMinute возвращает то же значение и помечено JsonIgnore, чтобы не создавать второе поле в JSON. На защите говори «знаки в минуту». Формулы слов в минуту здесь нет.

Как создаётся главное окно

Конструктор без параметров MainWindow() вызывает другой конструктор и передаёт new AppDataStore(). Конструктор с AppDataStore нужен в том числе тестам: они могут подставить хранилище во временной папке, не трогая настоящую историю пользователя.

Далее InitializeComponent создаёт элементы из AXAML. Окно читает настройки и статистику, загружает список текстов, выбирает первый и назначает обработчики. Например, StartButton.Click связан с StartSession, InputBox.TextChanged — с OnInputChanged. Таймер Tick вызывает UpdateMetrics каждые 250 миллисекунд.

Поля с подчёркиванием — внутренние данные окна. _store — хранилище; _session — текущая попытка; _timer — периодическое обновление экрана; _keys — соответствие символов изображениям клавиш; _statistics — история в памяти; _settings — применённые настройки. Это имена, выбранные разработчиком, а не особые команды C#.

Что происходит при нажатии Начать

StartSession проверяет, что в списке выбран WordDictionary. Затем очищает поле ввода и вызывает _session.Start(text, _settings.CaseSensitive). В сессии сохраняются образец и его название, обнуляется ввод, запоминается момент старта и устанавливается IsActive = true.

После этого окно разрешает редактирование InputBox, блокирует смену текста и регистра, меняет кнопку на «Заново», запускает таймер, перерисовывает образец и устанавливает фокус в поле ввода. Отсчёт начинается именно после нажатия «Начать», а не после первой буквы. Если задуматься перед вводом, это время повлияет на скорость.

Флаг _updatingInput временно поднимается при очистке поля программой. Изменение Text вызывает TextChanged даже тогда, когда текст изменил код. Без защитного флага собственное обновление окна могло бы попасть в обработку пользовательского ввода.

Проверка каждого изменения текста

Событие TextChanged передаёт управление OnInputChanged. Метод сразу выходит, если идёт внутреннее обновление или сессия не активна. В остальных случаях он передаёт всё текущее содержимое поля в UpdateInput, перерисовывает образец и пересчитывает показатели.

```

public bool UpdateInput(string input)
{
    if (!IsActive) return false;
    Input = input;
    if (Input.Length < Target.Length) return false;
    _finishedElapsed = Elapsed;
    IsActive = false;
    IsComplete = true;
    return true;
}

```

Это реальный метод `TypingSession`. Первая проверка запрещает обновлять уже завершённую попытку. `Input = input` сохраняет текущую строку. Пока она короче образца, результат `false`. Как только длина достигает длины образца или превышает её, время фиксируется и метод один раз возвращает `true`.

`True` здесь означает «именно сейчас произошёл переход к завершению», а не «текст набран без ошибок». Поэтому неверно набранный текст тоже завершает попытку. Повторный вызов уже вернёт `false`: `IsActive` выключен. Это предотвращает повторное добавление результата в историю.

После завершения поле становится только для чтения. Ошибку в последнем символе уже нельзя исправить в этой попытке: длина достигнута, результат зафиксирован. Можно начать заново. До завершения `Backspace` и исправления допускаются.

Сравнение символов без сложных слов

Сравнение позиционное: первый введённый символ сравнивается с первым символом образца, второй — со вторым и так далее. При `CaseSensitive = true` используется обычное равенство `char`. При `false` оба символа преобразуются через `char.ToUpperInvariant`, и уже затем сравниваются.

```

public bool Matches(char left, char right) => CaseSensitive
    ? left == right
    : char.ToUpperInvariant(left) == char.ToUpperInvariant(right);

```

Если регистр учитывается, «А» и «а» различаются. Если нет — считаются совпадением. Это простое посимвольное сравнение, а не лингвистический анализ слов. Пробелы и пунктуация тоже занимают позиции и должны совпадать.

Если в начале пропустить одну букву, последующие символы могут оказаться на неверных позициях и дать много ошибок. Алгоритм не ищет минимальное количество вставок и удалений, как редакционное расстояние Левенштейна. Такой выбор упрощает проверку и делает связь с ожидаемой следующей клавишей очевидной.

Формулы которые нужно уметь объяснить

Пусть N — длина текущего ввода, C — число совпадений по позициям, T — прошедшие секунды. Тогда ошибки $E = N - C$. Точность $A = C \times 100 / N$, а при $N = 0$ принимается 100%. Скорость $V = C \times 60 / \max(T, 1)$. Единица скорости — правильно набранные знаки в минуту.

```

var correct = 0;
for (var i = 0; i < Math.Min(Input.Length, Target.Length); i++)
    if (Matches(Input[i], Target[i])) correct++;

```

Это часть `GetStatistics`. `Math.Min` берёт меньшую длину, чтобы не выйти за границы одной из строк. Счётчик `correct` увеличивается только при совпадении. Все лишние введённые символы остаются в `Input.Length`, но не добавляют `correct`, поэтому считаются ошибками.

В выражениях 100d и 60d буква d задаёт double, чтобы расчёт был дробным. Math.Max(seconds, 1) заменяет слишком маленькое время одной секундой и предотвращает деление на ноль. Поэтому результат за половину секунды считается с нижней границей знаменателя в одну секунду.

Пример 1. Образец «котик», введено «ко», прошло 10 секунд. $N = 2$, $C = 2$, $E = 0$, $A = 100\%$, $V = 2 \times 60 / 10 = 12$ зн/мин. Остальные три ещё не введённых символа не считаются ошибками.

Пример 2. Образец «кот», введено «кит», прошло 6 секунд. $N = 3$, $C = 2$, $E = 1$, $A \approx 66,7\%$, $V = 20$ зн/мин. Длина достигнута, поэтому попытка завершится с одной ошибкой.

Пример 3. Образец «abcd», введено «ах». Одна ошибка и 50% точности. Пользователь нажал Backspace: осталось «а», теперь ноль ошибок и 100%. Ошибки отражают текущее содержимое, а не полную историю неправильных нажатий. Исправленная опечатка исчезает из счётчика, хотя потраченное время остаётся.

Пример 4. Образец «abc», одним изменением введено «abcх». $N = 4$, $C = 3$, $E = 1$, $A = 75\%$. Избыточный символ учитывается, а попытка завершается. Вставка из буфера специально не запрещена, поэтому приложение не является защищённым экзаменационным измерителем скорости.

Время и таймер выполняют разные задачи

TypingSession использует TimeProvider и временные метки GetTimestamp/GetElapsedTime для длительности. Когда попытка заканчивается, длительность записывается в _finishedElapsed. В дальнейшем она не растёт. Для даты результата используется локальная дата из источника времени.

DispatcherTimer с интервалом 250 мс нужен для обновления надписей на экране. Приложение не прибавляет 0,25 к длительности на каждом тике. Это различие полезно: задержка перерисовки не должна сама по себе менять источник измерения времени.

TimeProvider можно передать снаружи. В тестах есть свой Clock: проверка вручную «переводит» его на 10 или 30 секунд, не заставляя человека ждать. Это внедрение зависимости через конструктор. Отдельный контейнер внедрения зависимостей здесь не нужен.

Подсветка текста и клавиатуры

RenderTarget создаёт отдельный Run для каждого символа образца. Run — фрагмент текста со своим оформлением. Правильно введённые символы становятся зелёными, неверные — красными. Следующий ожидаемый символ выделяется синим и подчёркивается. После вычисления расположения текста окно прокручивает образец к текущей позиции.

BuildKeyboard строит ряды элементов Border с подписями. _keys хранит связь между символом и соответствующим Border. HighlightNextKey берёт Target[Input.Length] и окрашивает нужную клавишу. Для заглавной буквы при учёте регистра подсвечивается Shift; в русской подсказке Shift также используется для запятой.

Это визуальная подсказка, а не экранная клавиатура: на Border не назначены обработчики, которые вводят букву по щелчку. Раскладка подсказки выбирается по названию текста, начинающемуся с «Text». Приложение не переключает раскладку самой операционной системы. Соответствие клавиш упрощено и не является полным определением всех раскладок.

Настройки и история на диске

AppDataStore использует папку LocalApplicationData/TypingTrainer. В Windows её удобно открыть через %LOCALAPPDATA%\TypingTrainer. Там settings.json содержит настройки, statistics.json — массив результатов. При недоступном пустом системном пути предусмотрен запасной корень рядом с приложением. Сервер и СУБД для этой задачи не применяются.

JSON — текстовая запись объектов: имена свойств и их значения. Сериализация превращает объект C# в JSON, десериализация восстанавливает объект из JSON. System.Text.Json выполняет эти преобразования. WriteIndented = true добавляет отступы для удобного чтения человеком.

При сохранении сначала создаётся уникальный временный файл рядом с целевым. Затем File.Move с overwrite: true заменяет целевой файл подготовленным содержимым. finally удаляет оставшийся временный файл. Это уменьшает риск оставить обрезанный файл при ошибке записи. Здесь нет полноценной транзакционной базы или гарантии сохранности при любых аппаратных сбоях.

Если JSON синтаксически повреждён, загрузка возвращает значение по умолчанию. Размер шрифта ограничивается диапазоном 10–30 и проверяется на допустимое конечное число. Ошибки доступа и обычные ошибки файлов окно показывает в StatusText. В текущем коде нет полной проверки всех полей чужого JSON: файл истории предполагается созданным приложением.

История при сохранении сортируется по дате и ограничивается 500 результатами. На экране есть количество попыток, лучшая скорость и средняя точность по попыткам. Average здесь даёт обычное арифметическое среднее процентов, а не точность по сумме всех набранных символов.

«Применить» сначала сохраняет настройки и только после успешной записи обновляет используемый объект настроек. Во время активной попытки изменение режима регистра заблокировано. Очистка статистики требует отдельного подтверждения; память очищается после успешной записи пустой истории.

Notebook для анализа

Файл typing-trainer-notebook.ipynb — отдельный документ с пояснениями и выполняемыми ячейками C#. Он читает statistics.json, выводит количество попыток, лучшую скорость, среднюю точность, сумму ошибок и группирует результаты по текстам. GroupBy объединяет попытки с одинаковым названием текста.

Кнопка «Открыть notebook» передаёт файл программе, назначенной в системе для его открытия. Она не встраивает среду исполнения C# в окно тренажёра. Для работы по инструкции notebook нужен VS Code с Polyglot Notebooks и ядром C#. Эти дополнительные инструменты понадобятся только для notebook, а не для основной тренировки.

Для демонстрации сначала заверши хотя бы одну попытку, затем открой notebook и выполняй ячейки сверху вниз. Если результатов нет, он сообщает об этом. В его коде имя WordsPerMinute также трактуется как знаки в минуту.

4 Проверка запуск и изменения тренажёра

Что реально проверено

11 сентября 2026 года выполнен существующий набор TypingTrainer.Tests: 15 пройденных тестовых случаев, 0 ошибок, 0 пропусков. Используются имеющиеся исходники. Тесты не доказывают отсутствие всех дефектов и не заменяют репетицию на компьютере защиты.

CoreTests проверяет, что невведённые символы не являются ошибками; регистр работает для русского и английского; Backspace пересчитывает текущие ошибки; завершение происходит один раз; время после завершения не растёт; лишние символы считаются ошибками; каталог содержит нужные десять текстов; данные переживают повторную загрузку; история ограничена; C#-ячейки notebook компилируются и выполняются в тестовой проверке.

WindowTests через Avalonia.Headless проверяет выбор текста, ввод, завершение, повторное чтение результата, настройки, подтверждение очистки, прокрутку и размеры 1240 × 800 и 1000 × 700. Headless означает тестовую платформу для управления интерфейсом без обычного ручного сеанса. Это проверка окна, а не только математических функций.

Тест устроен по схеме «подготовить → выполнить → проверить». Например: создать сессию с тестовыми часами → ввести «abc» через 10 секунд → Assert.Equal проверяет 18 зн/мин и зафиксированное время. [Fact] обозначает отдельный тест, [Theory] — тест с наборами параметров InlineData.

Как запустить из исходников

В TypingTrainer.csproj указаны net9.0 и Avalonia 12.1.2. Это версии проекта, а не утверждение о самых новых выпусках. Для сборки нужны подходящий .NET SDK и доступные NuGet-пакеты. Команды ниже выполняются из корня папки с TypingTrainerApp.slnx.

```
dotnet restore TypingTrainerApp.slnx
dotnet run --project TypingTrainer/TypingTrainer.csproj
dotnet test TypingTrainer.Tests/TypingTrainer.Tests.csproj
```

restore получает зависимости; run собирает при необходимости и запускает приложение; test запускает проверки. Для готовой сборки под Windows x64 можно выполнить следующую команду. Это рецепт упаковки, а не заявление о проверке каждого возможного компьютера.

```
dotnet publish TypingTrainer/TypingTrainer.csproj -c Release -r win-x64 --self-contained true -o
outputs/trainer-publish
```

Для переноса копируют всю папку публикации. notebook включён в правила копирования проекта. Проблема «не найден SDK» решается установкой подходящего SDK; проблема получения пакетов — проверкой источников NuGet и сети; проблема открытия notebook — настройкой программы для .irupb. Это разные этапы, их нельзя смешивать.

Куда вносить небольшие изменения

Добавить тренировочный текст: AppDataStore.CreateDefaultTexts и тест каталога. Изменить формулу скорости: TypingSession.GetStatistics, подписи интерфейса, формат результата при необходимости, notebook и связанные тесты. Изменить цвета подсветки: кисти CorrectBrush, ErrorBrush и CurrentBrush в MainWindow.axaml.cs.

Изменить расположение панели: MainWindow.axaml. Изменить лимит истории: SaveStatistics и ограничение списка после завершения в MainWindow. Изменить диапазон шрифта: Slider в AXAML и Math.Clamp в AppDataStore. Для защиты важно понимать, что связанное правило иногда задано и в интерфейсе, и в проверке данных.

Ограничения и возможное развитие

Нельзя заявлять, что приложение считает каждое ошибочное нажатие за всю попытку, проверяет слова по смыслу или автоматически переключает раскладку. Оно сравнивает текущую строку по позициям и выдаёт

подсказку. Счётчик `char` подходит для обычных встроенных текстов, но не равен числу видимых символов во всех языках.

Завершение по длине позволяет закончить с ошибками и блокирует исправления после достижения длины. Вставка текста не запрещена. Каталог фиксированный. Нет пользователей, облачной синхронизации, базы SQL и многопользовательского рейтинга. Сохранение локальное, шифрование истории отдельно не реализовано.

Разумные направления развития: режим завершения только после полного совпадения; отдельный счётчик всех ошибочных нажатий; редактор собственных текстов; более точная модель клавиатуры; графики прогресса; выделение `ViewModel`. Перед изменением режима подсчёта нужно определить, какие результаты в старой истории останутся сопоставимыми.

5 Подготовка к защите тренажёра

Речь примерно на три минуты

«Мой проект — настольный тренажёр набора текста на C# и Avalonia. Он позволяет выбрать один из десяти русских и английских текстов, перепечатать его и получить время, скорость в правильно набранных знаках в минуту, точность и количество ошибок.

Приложение разделено на три проекта. `TypingTrainer` содержит окно и обработчики событий. `TypingTrainer.Core` содержит модели, логику попытки и файловое хранилище. `TypingTrainer.Tests` содержит автоматические проверки. В этой версии управление окном выполнено через `code-behind`; отдельной `ViewModel` у тренажёра нет.

При нажатии «Начать» создаётся состояние попытки и фиксируется момент старта. Каждое изменение поля ввода передаёт текущую строку в `TypingSession`. Символы сравниваются с образцом по одинаковым позициям. Проверка регистра настраивается. Правильные символы подсвечиваются зелёным, неправильные красным, следующая позиция выделяется отдельно.

Скорость вычисляется как число правильных символов, умноженное на 60 и разделённое на прошедшие секунды с нижней границей в одну секунду. Точность — доля правильных символов среди уже введённых. Исправленные опечатки исчезают из счётчика ошибок, поскольку проверяется текущее содержимое. Попытка завершается при достижении длины образца, даже если есть ошибки.

Настройки и до 500 результатов сохраняются локально в JSON. Для записи сначала готовится временный файл. Отдельный `notebook` на C# позволяет анализировать историю. Логiku можно проверять без окна благодаря разделению проектов и подменяемому источнику времени.

Существующий набор автоматических тестов прошёл 15 случаев. Среди ограничений — фиксированные тексты, отсутствие защиты от вставки и упрощённая клавиатура-подсказка. Дальше можно добавить собственные тексты и отдельный режим строгого завершения. Сейчас покажу полный цикл тренировки и сохранённый результат».

План показа на экране

1. Запусти программу и покажи список из десяти текстов. Назови элементы окна и единицу скорости.
2. Поставь удобный размер шрифта, нажми «Применить». Покажи, что настройка действительно изменила образец и поле ввода.

3. Начни попытку. Введи несколько правильных символов, затем опечатку и удали её до завершения. Объясни изменение цвета, ошибки и точности.
4. Покажи подсказку следующей клавиши и поясни, что системную раскладку выбирает пользователь. «Сброс» используй для демонстрации прекращения незавершённой попытки.
5. Заверши одну попытку. Открой «Статистику», затем перезапусти приложение и покажи сохранённый результат. Историю специально очищать для защиты не требуется.
6. В коде открой `TypingSession.UpdateInput` и `GetStatistics`. На примере «кот» и «кит» объясни завершение и формулы.
7. Покажи `AppDataStore.Save` и один тест с `Clock`. Notebook демонстрируй, только если заранее проверил его среду запуска.

Возможные вопросы и короткие ответы

Почему C# и Avalonia? C# даёт строгие типы, классы и библиотеку .NET; Avalonia позволяет описать окно разметкой и управлять им из C#. В данном проекте используется настольный жизненный цикл.

Где точка входа? `Program.Main`. Затем запускаются `App` и `MainWindow`. Разметка сама по себе не является первой выполняемой функцией.

Это MVVM? У тренажёра нет отдельной `ViewModel`. Есть `code-behind` окна и отдельно вынесенная логика `Core`. Я могу показать эту границу по файлам.

Зачем `Core`? Чтобы расчёты, данные и сохранение можно было использовать и тестировать отдельно от интерфейса.

Когда начинается и заканчивается измерение? После «Начать» и при достижении длины образца. Источник времени — `TimeProvider`; экран обновляет `DispatcherTimer`.

Почему скорость не слова в минуту? Формула использует правильные символы без деления на длину слова. `WordsPerMinute` — старое имя JSON-поля, а интерфейс показывает знаки в минуту.

Почему точность в начале 100%? Так определён случай пустого ввода, чтобы не делить на ноль. Это не означает, что весь образец уже набран.

Почему ошибка исчезает после `Backspace`? Статистика пересчитывается по текущей строке. События ошибочных нажатий отдельно не накапливаются.

Можно ли закончить с ошибками? Да. Завершение проверяет длину, а правильность отражается в статистике. Это нужно различать.

Почему нельзя изменить регистр посередине? Тогда одна попытка оценивалась бы по разным правилам. На время попытки настройка заблокирована.

Как избегаеете двойного результата? `UpdateInput` возвращает `true` только в момент перехода к завершению, после чего `IsActive` становится `false`.

Как хранятся результаты? Массив объектов сериализуется в `statistics.json` в локальной папке данных. Сохраняются до 500 последних результатов.

Что будет при ошибке сохранения? Окно показывает сообщение. Результат может оставаться в текущем списке в памяти, но нельзя считать его гарантированно записанным на диск.

Как тестировать время без ожидания? Передать тестовый `TimeProvider`, вручную сдвигать его метку и проверять формулу.

Можно ли обмануть измерение вставкой? Да, запрета вставки нет. Это учебный тренажёр, а не система контроля экзамена.

Какова сложность сравнения? Один расчёт проходит до $\min(N, M)$ символов, то есть линейно по длине. Перерисовка строит символы образца заново; для длинных текстов потребовалась бы оптимизация.

Самопроверка перед выходом

Закрой пособие и объясни своими словами: чем отличаются `C#`, `.NET` и `Avalonia`; где находится обработчик кнопки; что хранит `TypingSession`; почему невведённые символы не ошибки; как получаются 20 зн/мин в примере «кот» и «кит» за 6 секунд; зачем нужен `JSON` и тестовый `Clock`. Если можешь показать это в коде, основа защиты уже понятна.

Не заучивай слова, которые не можешь связать с файлом. Если забыл название метода, начни с действия: «После изменения текста окно передаёт строку в сессию; сейчас покажу этот участок». Это точнее, чем приписывать программе несуществующий алгоритм.

6 Где проверить объяснения

Главный источник описания приложения — его исходники. Имена классов и методов в тексте позволяют найти соответствующий участок поиском в редакторе кода. Версии библиотек указаны по `.csproj`, результаты тестов — по запуску 11 сентября 2026 года. Учебные фрагменты отдельно названы учебными; основные алгоритмы приведены из проекта.

`TypingTrainer.Core/TypingSession.cs` — состояние попытки, сравнение и формулы.

`TypingTrainer.Core/AppDataStore.cs` и `Models.cs` — сохранение и модели данных.

`TypingTrainer/MainWindow.axaml` и `MainWindow.axaml.cs` — окно и обработчики.

`TypingTrainer.Tests/CoreTests.cs` и `WindowTests.cs` — проверяемые сценарии.

`typing-trainer-notebook.ipynb` — анализ истории.

Официальные материалы для уточнения терминов

Microsoft Learn — обзор языка `C#`

<https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/>

Microsoft Learn — асинхронное программирование

<https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/>

Avalonia — контекст данных

<https://docs.avaloniaui.net/docs/data-binding/data-context>

Avalonia — синтаксис привязки

<https://docs.avaloniaui.net/docs/data-binding/data-binding-syntax>

Avalonia — MVVM

<https://docs.avaloniaui.net/docs/fundamentals/the-mvvm-pattern>